

↳ Simple threading

↳ Functions - with thread.

↳ parallel programming

↳ explicit parallel programming
(Task parallel library (TPL))

↳ Implicit parallel programming.

↳ Parallel for / foreach

↳ Parallel Loop (PLinq)

Garbage Collection Algorithm

Task $T_1 = \text{new Task}(f_1)$

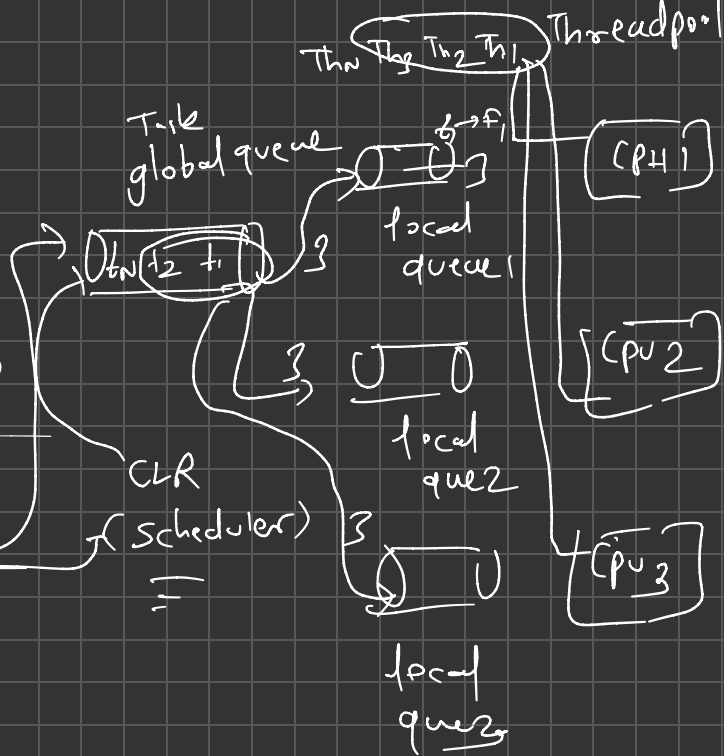
Task $T_2 = \text{new Task}(f_2)$

$T_n = \text{new Task}(f_n)$

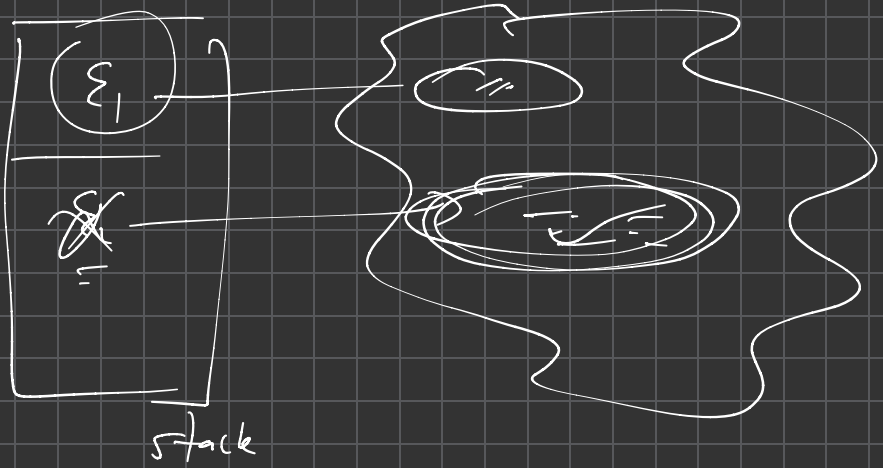
$T_1.\text{start}()$

$T_2.\text{start}()$

$T_n.\text{start}()$



$\text{Emp } \varepsilon_i = \text{new Emp}()$ | $\varepsilon_i = n+1$

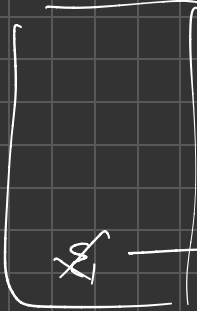


$\xi_1 = \text{new Expr}()$

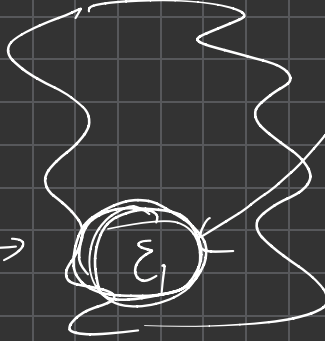
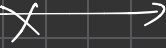


$\xi_{R.L.}$

gc tree
=

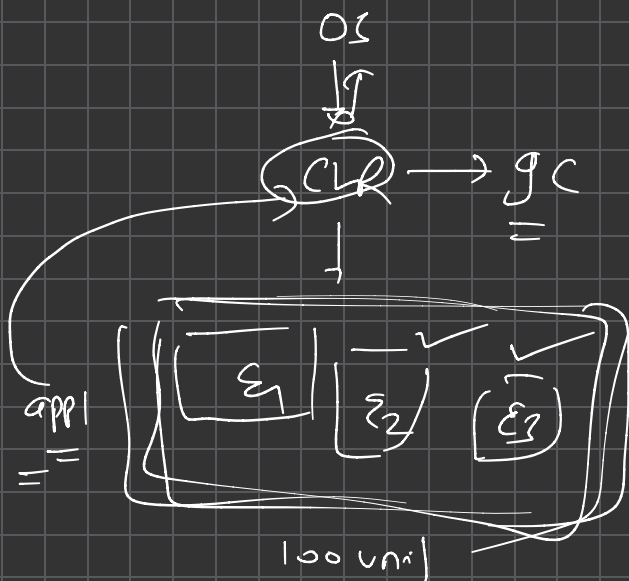


Stack



heap

$\xi_1 = \text{null}$
=

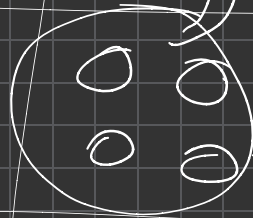


$\text{Emp } \epsilon_i = \text{new Emp},$

gen 2

(gen)

gen 0



~~gen 1~~

~~gen 1~~

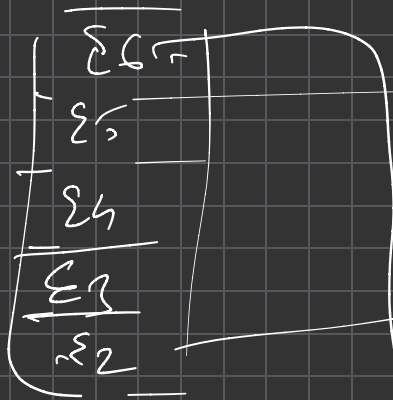
gc

—

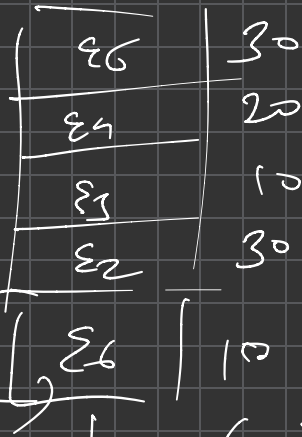
$\xi_1 = \text{new}$

ξ_2

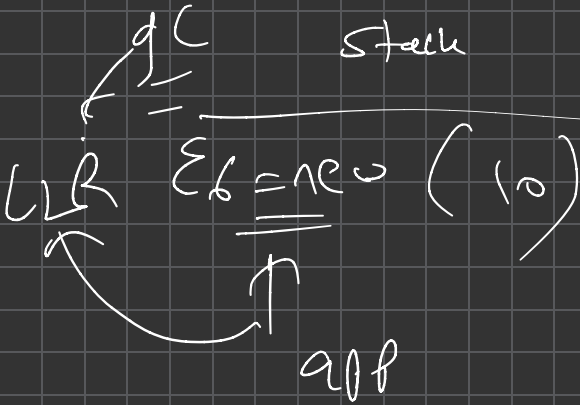
ξ_3



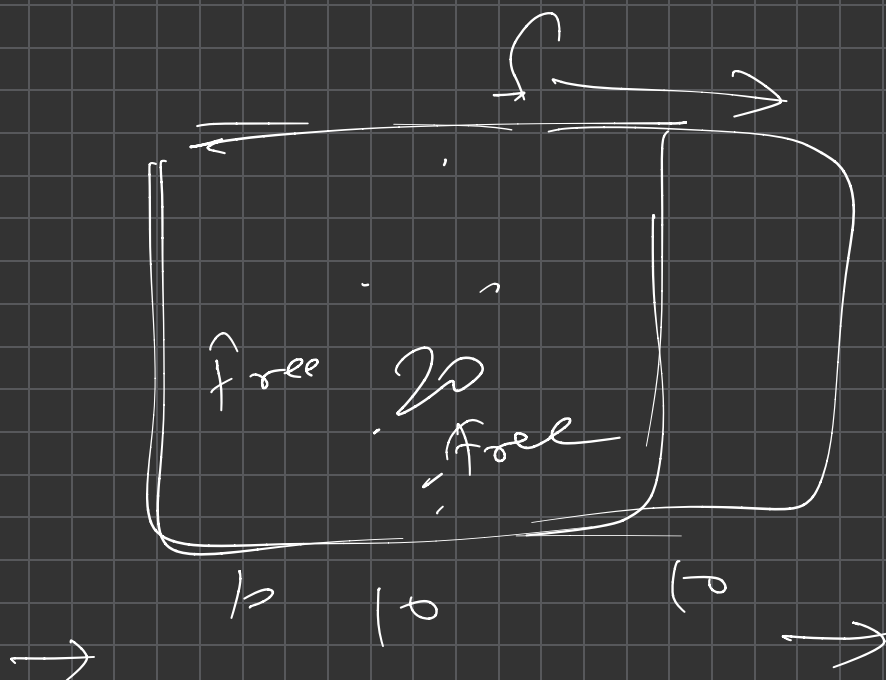
stack



heap (1000)



$\xi_1 = \text{null}$



$$\begin{array}{r}
 E_1 = 100 \\
 \hline
 20 \\
 \hline
 \hline
 \end{array}$$

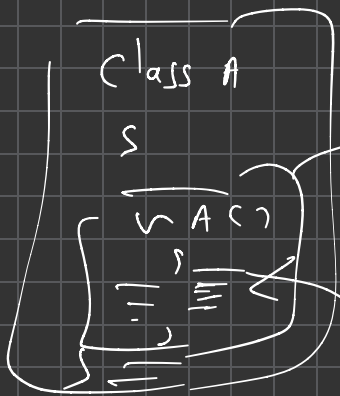
- ① Set object = null at earliest.
 - ② Avoid creating global / static objects
 - ③ Write small size functions & local scoped objects
Small size $\approx$ 2 to 4 lines code in function =
 - ④ Avoid creating Large objects
 - ⑤ Do not call "gc" by your own
 - ⑥ Avoid writing your own destructor
-

class Emp{

~Emp()

≡ code cleaning

xi = null



⚡
A obj = new A()
==

[obj = null] or

obj.delete()

call
destructor

out of scope (obj = null)

gc
↓
gc calls
destructor
←

